

FINAL LABORATORY EXAM

Course: Application Development and Emerging Technology

Total Points: 50 points

Exam Type: Hands-on Laboratory (Linux + Python + Git/GitHub + API using Postman)

Submission: GitHub Repository Link + Video Recording Link + Screenshots + JSON File

GENERAL EXAM INSTRUCTIONS

1. Use a **Linux environment** (Virtual Machine, WSL, or any Linux distribution).
 2. You must use **your own GitHub account**.
 3. Record **one continuous video** showing:
 - Terminal commands
 - File and folder creation
 - Python execution
 - Git operations
 - Postman API calls
 - Narration explaining what you are doing
 4. Perform tasks **in order**.
 5. All outputs must be **clearly visible** in the recording.
 6. Use **your own full name** in:
 - Shell script output
 - Python program input
 - Webex room names and messages
 7. Submit all required links and files **before the deadline**.
-

PART I — LINUX COMMAND LINE OPERATIONS (15 POINTS)

Task 1: Directory Structure (6 pts)

1. Create a directory named:

ADV_Midterm_

2. Inside it, create the following subdirectories:
 - scripts
 - python_program
 - git_project

Task 2: File Management (6 pts)

1. Inside ADV_Midterm_, create the following text files:
 - student_list.txt
 - courses.txt
 - subjects.txt
2. Using Linux commands:
 - Copy **one** of the text files
 - Rename **another** text file

Task 3: Shell Script (3 pts)

1. Inside the scripts directory, create a shell script named:
info.sh
2. The script must print:
 - Your **full name**
 - Current **date**
 - **System information** (OS / kernel / hostname)
3. Make the script executable and run it.

PART II — PYTHON SETUP & ENROLLMENT SYSTEM (20 POINTS)

Task 1: Python Setup (5 pts)

1. Verify Python installation:

python3 --version

2. If not installed, install Python and show verification.

Task 2: Enrollment System Program (15 pts)

1. Inside python_program, create:

enrollment_system.py

2. The program must:

- Accept student information:
 - Full Name
 - Address
 - Age
- Allow the user to:
 - Select a **course**
 - Select **subjects** (or units)
- Compute **total payment**:
 - Number of subjects or units × rate
- Display a **formatted enrollment summary** including:
 - Student details
 - Selected course
 - Subjects
 - Total payment

3. Run the program and show output in the video.

PART III — GIT & GITHUB OPERATIONS (15 POINTS)

Task 1: Repository Setup (3 pts)

1. Navigate to the git_project directory.
2. Initialize a Git repository:

git init

Task 2: Commit Python Program (3 pts)

1. Copy enrollment_system.py into git_project.
2. Add and commit the file with a meaningful message.

Task 3: Branching (3 pts)

1. Create a new branch:

feature-update

2. Make a small edit to the Python file.
3. Commit the changes.

Task 4: GitHub Push (6 pts)

1. Create a GitHub repository.
2. Push all branches to GitHub.
3. Show the repository in your browser.

PART IV — API LAB USING POSTMAN & CISCO WEBEX (FAKER REQUIRED) (50 POINTS)

**All API calls MUST be done using Postman
Cisco Webex REST API only**

A. POSTMAN ENVIRONMENT & AUTHENTICATION (10 PTS)

1. Log in to developer.webex.com and copy your **Personal Access Token**.
2. Create a Postman environment named:

SemiFinal_

3. Add variable:

- token = <your Webex token>
 - 4. Set Authorization:
 - Type: **Bearer Token**
 - Token: {{token}}
 - 5. Take screenshots of:
 - Environment variables
 - Authorization tab
-

B. ROOMS OPERATIONS (20 PTS)

B1: GET — List Rooms (5 pts)

- Endpoint:

GET <https://webexapis.com/v1/rooms>

- Screenshot request and response.

B2: POST — Create Room (5 pts)

- Room title format:

<Your Full Name> SemiFinal Room

- Save returned roomId to a Postman variable.
- Screenshot request and response.

B3: PUT — Update Room (5 pts)

- Update title to:

<Your Full Name> Updated SemiFinal Room

- Screenshot request and response.

B4: DELETE — Delete Room (5 pts)

- Delete the same room using roomId.
 - Screenshot request and response.
-

C. PYTHON + FAKER LIBRARY DEMONSTRATION (20 PTS)

1. Install Faker:

`pip3 install faker`

2. Open Python shell and demonstrate:

```
from faker import Faker
```

```
fake = Faker()
```

```
fake.name()
```

```
fake.catch_phrase()
```

```
fake.isbn13()
```

3. Show **tab completion** (fake. + TAB TAB) displaying Faker methods.
4. Create a Python script (example: add100RandomBooks.py) that:
 - Uses requests
 - Uses faker to generate:
 - Book title
 - Author name
 - ISBN
 - Prints at least **10 fake records**

REQUIRED SUBMISSIONS

1. GitHub repository link
2. Video recording link (continuous)
3. Screenshots of **every** Postman request and response
4. One JSON file containing **all POST and PUT request bodies**

GRADING NOTE

- Missing screenshots or incomplete video = **0 points** for that item
- Copy-pasted data = **automatic failure**
- Late submission = **no credit**